

The Context-Aware Sales Assistant: Empowering LLMs with MCP and Zapier"

Project Goal: To demonstrate the value of the Model Context Protocol (MCP) by enabling a Python-based LLM client to securely and dynamically access Zapier's vast ecosystem of integrations (acting as an MCP server), allowing the LLM to perform real-world actions and retrieve real-time data based on user queries, all through a standardized protocol.

Core Value Proposition of MCP Demonstrated:

- **Standardization:** A single client can interact with diverse Zapier "tools" without needing custom API integrations for each.
- **Dynamic Tool Discovery:** The LLM client can "discover" available tools and their schemas from the MCP server.
- **Contextual Action:** The LLM can intelligently decide *when* to use a tool and *what data* to provide, based on the user's request.
- **Real-world Impact:** AI agents move beyond just conversation to perform concrete actions (sending emails, checking CRM data, etc.).
- **Security (Implicit):** Zapier MCP handles authentication and authorization, providing a secure layer.

Scenario: A Sales Assistant Chatbot

Imagine a sales assistant whose capabilities are limited to what it was trained on. With MCP and Zapier, we can dramatically extend its reach.

User Prompt Examples:

1. "What was the last email I sent to John Doe at example.com?" (Requires accessing an email tool)
2. "Can you create a new lead in Salesforce for Sarah Connor, email sarah.c@example.com, company Cyberdyne?" (Requires accessing a CRM tool)
3. "Schedule a follow-up call with Mark Smith for next Tuesday at 10 AM regarding project Alpha." (Requires accessing a calendar tool)

Key Components & Steps:

1. **Zapier MCP Server Setup (Manual - Prerequisite for Demo):**

- Create a Zapier account.
- Go to <https://mcp.zapier.com/>
- Create a New MCP Server.
- **Add "Tools" to the server:**
 - **Gmail:**
 - Action: "Find Email" (for retrieving emails)
 - Action: "Send Email" (for sending follow-ups)
 - **Salesforce (or Google Sheets for simplicity):**
 - Action: "Find Record" (e.g., find a Lead by email)
 - Action: "Create Record" (e.g., create a new Lead)
 - **Google Calendar:**
 - Action: "Find Event"
 - Action: "Create Event"
- **Crucially, obtain the MCP Server URL.** This URL will be used by our Python client. (Note: Zapier MCP is free for up to 300 tool calls/month, which is sufficient for a demo).

2. Python Client (Hugging Face `huggingface_hub` MCP Client):

- We'll use `huggingface_hub`'s MCP client, as it's designed for LLM tool integration.
- This client will connect to the Zapier MCP Server.
- It will expose the discovered Zapier tools to a local LLM (e.g., a small instruct model like Phi-2).

3. LLM (Local - e.g., Phi-2 Instruct):

- The LLM will be responsible for understanding the user's intent and deciding which Zapier MCP tool to call, and with what arguments. This relies on the LLM's function-calling/tool-use capabilities.

(You can try this with Amazon Bedrock LLM as well)

Mini-Project Structure & Code (Conceptual / Pseudocode for Zapier Integration)

Colab Setup (Initial Cells):

```
# Install necessary libraries
```

```
!pip install -q transformers accelerate bitsandbytes peft torch
```

```
!pip install -q huggingface_hub[mcp] # MCP client from Hugging Face
```

```
!pip install -q python-dotenv # For securely loading API keys
```

```
# Authenticate with Hugging Face Hub (for model access)
```

```
# from huggingface_hub import login
```

```
# login() # Or set HF_TOKEN environment variable
```

```
# Ensure your Zapier MCP Server URL is set as an environment variable or loaded securely
```

```
# For demo purposes, you might hardcode it if privacy isn't a concern, but NOT for production
```

```
# import os
```

```
# os.environ["ZAPIER_MCP_SERVER_URL"] =  
"YOUR_ZAPIER_MCP_SERVER_URL_HERE"
```

1. Define LLM & Tokenizer

```
from transformers import AutoModelForCausalLM, AutoTokenizer, pipeline
```

```
import torch
```

```
model_id = "microsoft/Phi-2" # Or a small instruct model like 'HuggingFaceH4/zephyr-7b-beta' if you have compute
```

```
tokenizer = AutoTokenizer.from_pretrained(model_id, trust_remote_code=True)
```

```
tokenizer.pad_token = tokenizer.eos_token # Essential for batch inference if used
```

```
# Load the base model (you can use 4-bit quantization to save memory)
```

```

model = AutoModelForCausalLM.from_pretrained(
    model_id,
    torch_dtype=torch.bfloat16, # Or torch.float16 or 4-bit/8-bit if memory is an issue
    device_map="auto",
    trust_remote_code=True
)

```

Create a text generation pipeline

```

llm_pipe = pipeline(
    "text-generation",
    model=model,
    tokenizer=tokenizer,
    device_map="auto"
)

```

Helper function for LLM generation with tool calls

```

def generate_llm_response(pipe, prompt, tools_schema=None):

```

```

    """

```

```

    Generates a response from the LLM, potentially including tool calls.

```

```

    The 'tools_schema' is critical for the LLM to understand available tools.

```

```

    """

```

```

    messages = [
        {"role": "system", "content": "You are a helpful sales assistant. Use the available
tools to answer questions or perform actions."},
        {"role": "user", "content": prompt}
    ]

```

This is a simplified representation. Actual tool calling involves:

1. LLM generating a tool call.
2. Client executing the tool.
3. Client re-prompting the LLM with tool output.
The Hugging Face `MCPClient` handles this orchestration.

If tools_schema is provided, include it in the prompt (e.g., via function calling spec or few-shot examples)

For models that natively support function calling (like some OpenAI, Anthropic models), you'd pass tools_schema

directly to the generation method. For simpler models like Phi-2, you'd integrate it into the prompt.

This is a placeholder for the actual tool-calling integration logic

The Hugging Face MCPClient will wrap this.

For a direct LLM-only approach for simple models, you'd describe the tools in the system prompt.

```
system_prompt = "You are a helpful sales assistant. You have access to the following tools via the Model Context Protocol (MCP):\n"
```

```
if tools_schema:
```

```
    for tool_name, tool_desc in tools_schema.items():
```

```
        system_prompt += f"- Tool: {tool_name}\n Description: {tool_desc['description']}\n Parameters: {tool_desc['parameters']}\n"
```

```
        system_prompt += "When a user asks you to perform an action or retrieve specific real-time information, *output a JSON object* containing the `tool_name` and `tool_arguments` you would call. If you cannot fulfill the request with a tool, respond naturally.\n"
```

```
    else:
```

```
        system_prompt += "You do not have access to external tools. Answer based on your knowledge.\n"
```

```
messages[0]["content"] = system_prompt + messages[0]["content"] # Prepend tools
to system instruction
```

```
chat_template = tokenizer.apply_chat_template(messages, tokenize=False,
add_generation_prompt=True)
```

```
output = pipe(
    chat_template,
    max_new_tokens=250,
    do_sample=True,
    temperature=0.7,
    top_k=50,
    top_p=0.95,
    repetition_penalty=1.1,
    eos_token_id=tokenizer.eos_token_id,
    pad_token_id=tokenizer.eos_token_id
)
generated_text = output[0]['generated_text']
# Extract only the assistant's part
if "<|assistant|>" in generated_text:
    return generated_text.split("<|assistant|>")[-1].strip()
return generated_text.strip()
```

2. MCP Client Integration (Core of the Demo)

```
from huggingface_hub import InferenceClient
from huggingface_hub.mcp import MCPClient
import json
import os
```

```

import re

# Load Zapier MCP Server URL from environment variable
# Make sure to set this before running, e.g., in Colab:
# import os

# os.environ["ZAPIER_MCP_SERVER_URL"] =
"YOUR_ZAPIER_MCP_SERVER_URL_FROM_ZAPIER_MCP_DASHBOARD"

zapier_mcp_url = os.getenv("ZAPIER_MCP_SERVER_URL")
if not zapier_mcp_url:
    raise ValueError("ZAPIER_MCP_SERVER_URL environment variable not set. Please set
it from your Zapier MCP dashboard.")

# Initialize the Hugging Face InferenceClient (this is the LLM client)
# This client will also act as the MCP client manager.
inference_client = InferenceClient(
    model="microsoft/Phi-2", # Or the model you loaded
    token=os.getenv("HF_TOKEN") # Your Hugging Face API token for model access
)

# Initialize the MCPClient. This is the crucial part that connects to the Zapier server.
# The `servers` parameter defines how to connect to the MCP server.
# For Zapier, it's typically an SSE (Server-Sent Events) HTTP transport.
mcp_client = MCPClient(
    inference_client=inference_client, # Link to your LLM inference client
    servers=[
        {
            "type": "sse",
            "url": zapier_mcp_url,

```

```

        "name": "Zapier_MCP_Server" # A friendly name for your server
    }
]
)

# --- Dynamic Tool Discovery ---

# Get available tools from the Zapier MCP server

# This is a key feature of MCP: the client doesn't need to be hardcoded with tool info.
print("\n--- Discovering Tools from Zapier MCP Server ---")

available_tools = mcp_client.get_tools()

tools_schema = {}

print("Available tools:")

for tool in available_tools:
    print(f"- Name: {tool.name}")
    print(f" Description: {tool.description}")

    # MCP defines standardized JSON schema for parameters
    print(f" Parameters: {tool.input_schema.json_schema()}")

    tools_schema[tool.name] = {
        "description": tool.description,
        "parameters": tool.input_schema.json_schema()
    }

print("-" * 50)

# --- Orchestration Function (Simulating an Agent's Loop) ---

async def sales_agent_chat(user_query: str):
    print(f"\nUser: {user_query}")

    # Step 1: LLM decides if a tool is needed based on the query and available tools

```

```

# We pass the tools_schema to the LLM's prompt so it knows what it can do.

# In a real tool-calling LLM, the LLM would naturally output the tool call.

# For simpler LLMs, we'll guide it with prompt engineering to output JSON.

llm_initial_response_text = generate_llm_response(llm_pipe, user_query,
tools_schema)

print(f"LLM's initial thought/output: {llm_initial_response_text}")


# Check if the LLM decided to call a tool (looking for a JSON structure)

tool_call_match = re.search(r"```json\s*(\{.*?\})\s*```", llm_initial_response_text,
re.DOTALL)


if tool_call_match:

    try:

        tool_call_json = json.loads(tool_call_match.group(1))

        tool_name = tool_call_json.get("tool_name")

        tool_args = tool_call_json.get("tool_arguments", {})


        if tool_name and tool_args:

            print(f"--- LLM identified tool call: {tool_name} with args: {tool_args} ---")

            # Step 2: Execute the tool via MCPClient

            try:

                tool_result = await mcp_client.call_tool(tool_name, tool_args)

                print(f"--- Tool '{tool_name}' executed. Result: ---")

                print(tool_result)


            # Step 3: Feed tool results back to LLM for final response generation

            contextual_prompt = f"The user asked: '{user_query}'. I called the tool
'{tool_name}' with arguments {tool_args} and got the following

```

```
result:\n{json.dumps(tool_result, indent=2)}\n\nNow, formulate a polite and concise response to the user based on this information."
```

```
    final_llm_response = generate_llm_response(llm_pipe, contextual_prompt)

    print(f"Agent's final response: {final_llm_response}")

    return final_llm_response

except Exception as e:

    print(f"Error executing tool {tool_name}: {e}")

    return f"I encountered an error trying to perform that action: {e}. Please try again."

else:

    print("LLM tried to call a tool but missing name or arguments.")

    return f"I understood you want to use a tool, but I couldn't parse the tool call. Please try rephrasing."

except json.JSONDecodeError:

    print("LLM's tool call was not valid JSON.")

    return "I tried to use a tool, but there was an issue understanding the instructions. Can you clarify?"

else:

    print("LLM decided no tool was needed or generated a natural response.")

    print(f"Agent's natural response: {llm_initial_response_text}")

    return llm_initial_response_text
```

Demo Execution

```
import asyncio
```

```
# --- Test Cases ---
```

```
async def run_demos():
```

```

print("--- Demonstrating MCP Value ---")

# Demo 1: Retrieve information (Find Email)

await sales_agent_chat("What was the last email I sent to John Doe at
example.com?")

# Expected: LLM identifies 'find_email' tool, calls it, and summarizes the result.


# Demo 2: Perform an action (Create Lead/Record)

await sales_agent_chat("Can you create a new lead in Salesforce for Sarah Connor,
email sarah.c@cyberdyne.com, company Cyberdyne Systems?")

# Expected: LLM identifies 'create_record' tool, calls it, and confirms creation.

# Note: Replace 'Salesforce' with 'Google Sheets' if using that for simplicity.


# Demo 3: Perform an action (Schedule Event)

await sales_agent_chat("Schedule a follow-up call with Mark Smith for next Tuesday
at 10 AM regarding project Alpha.")

# Expected: LLM identifies 'create_event' tool, calls it, and confirms event creation.


# Demo 4: General knowledge (no tool needed)

await sales_agent_chat("What are the key benefits of using AI in sales?")

# Expected: LLM responds from its internal knowledge without calling a tool.


# Run the asynchronous demo

await run_demos()

```

PRESENTATION AND DEMO SCRIPT

Demo Title: "Context-Aware AI: Bridging LLMs to the Real World with Model Context Protocol (MCP) and Zapier"

Narrative Flow:

1. Introduction: The LLM Limitation:

- "LLMs are powerful, but they often lack real-time data and the ability to take actions. They're typically confined to their training data."
- "Imagine a sales assistant that can only *talk* about sales, but can't *do* anything practical like sending an email or updating a CRM."

2. Introducing the Problem: Fragmented Integrations:

- "Historically, integrating LLMs with external tools (like Salesforce, Gmail, Calendar) meant custom API coding for *each* service. This is complex, time-consuming, and brittle."

3. The Solution: Model Context Protocol (MCP):

- "Enter the **Model Context Protocol (MCP)**. This open standard is like a universal adapter for AI. It allows AI clients (our LLM) to discover and interact with external services (our Zapier integration) in a standardized way."
- "**Key Value 1: Standardization:** With MCP, we don't need a separate API wrapper for Gmail, Salesforce, and Google Calendar. Zapier acts as an MCP server, presenting all these tools via a single, consistent protocol."

4. Zapier as the MCP Server (Brief Setup Overview):

- "To make this practical, we're using **Zapier MCP**. Zapier is already connected to thousands of apps. By enabling it as an MCP server, we instantly gain access to a massive toolkit."
- "Show a quick screenshot or explain how you set up the Zapier MCP server and added the Gmail, Salesforce/Sheets, and Calendar actions. **Emphasize obtaining the ZAPIER_MCP_SERVER_URL.**"

5. The Python Client & LLM (Show Code Structure):

- "Our Python client, leveraging huggingface_hub's MCP capabilities, is the brain here. It loads a small LLM (like Phi-2) and connects to our Zapier MCP server."
- "**Key Value 2: Dynamic Tool Discovery:** Watch as our client first queries the Zapier MCP server to *discover* what tools are available and their exact parameters. The LLM *doesn't* need to know these upfront!" (Show the print(available_tools) output).

6. The Live Demo: Contextual Action in Action!

- **Run Demo 1 (Find Email):**

- "User asks: 'What was the last email I sent to John Doe at example.com?'"
- *Narrate the console output:* "Notice how the LLM, given the context of available tools, intelligently decides to call the find_email tool. The MCP client then executes this action via Zapier. And here's the real-time result, summarized beautifully by the LLM!"
- **Run Demo 2 (Create Lead):**
 - "User asks: 'Can you create a new lead in Salesforce for Sarah Connor, email sarah.c@cyberdyne.com, company Cyberdyne?'"
 - *Narrate:* "Again, the LLM correctly identifies the create_record (or similar) tool and extracts the necessary details – Sarah Connor, her email, her company. The MCP handles the secure invocation, and we get confirmation of lead creation!"
- **Run Demo 3 (Schedule Event):**
 - "User asks: 'Schedule a follow-up call with Mark Smith for next Tuesday at 10 AM regarding project Alpha.'"
 - *Narrate:* "This shows how the LLM can parse temporal information and specific context to correctly use the create_event tool, making our AI truly actionable."
- **Run Demo 4 (No Tool Needed):**
 - "User asks: 'What are the key benefits of using AI in sales?'"
 - *Narrate:* "Crucially, the LLM also knows *when not to use a tool*. For general knowledge questions, it simply responds from its own capabilities, demonstrating efficient resource use."

7. Conclusion: The Power and Value of MCP:

- "This mini-project demonstrates the profound value of the Model Context Protocol."
- "It transforms a conversational LLM into an **intelligent agent** that can perform real-world tasks and access dynamic data."
- "MCP provides the **standardization, security, and dynamic discovery** necessary for AI agents to seamlessly integrate with vast ecosystems of tools, moving us closer to truly autonomous and highly capable AI applications."

- "This is a fundamental shift in how we build AI, enabling capabilities that were previously complex or impossible."

References/Examples:

Build an MCP-powered financial analyst (100% local):

https://x.com/akshay_pachaar/status/1927372063259377984

Building your own MCP Server:

https://x.com/_avichawla/status/1929425316520632442?t=ZNODGhHEOswwo7vHdDUpUg&s=19